

```

In [1]: # =====
# ■ NAS100 Bot – Basse Fréquence / Forte Asymétrie (M15) — v2.0
# =====
# CORRECTIFS v2.0 :
# [FIX-1] Entrée Limit Pullback : entry = Close ± ATR×0.5 (vs market)
# [FIX-2] Breakeven inconditionnel : o['sl']=entry après TP1 (sans
condition)
# [FIX-3] Time-Stop 45 min : liquidation si TP1 non atteint
# [FIX-4] Filtre Vitesse : corps bougie ≥ 60% range (anti-
doji)
# [FIX-5] Sensitivity worker : tp_v is not None avant px>=tp_v
# [FIX-6] Cache ticks + timestamps : permet le Time-Stop tick-exact
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings, time, pickle, multiprocessing
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import timedelta
from pathlib import Path
from collections import OrderedDict
from tqdm.notebook import tqdm

try:
    import polars as pl
    print(f'✅ Polars {pl.__version__} disponible')
except ImportError:
    raise ImportError('pip install polars')

warnings.filterwarnings('ignore')
plt.style.use('dark_background')

# — Chemins —————
PATH_M15 = Path('data/NAS100._M15_1Year.csv')
PATH_M1 = Path('data/NAS100._M1_1Year.csv')
PATH_TICKS = Path('data/NAS100._Ticks_1Year.csv')
PATH_TICKS_PARQ = Path('data/NAS100._Ticks_1Year.parquet')
CACHE_IND = Path('data/nas100_m15_indicators_v2.pkl')

# — Contractuel —————
CONTRACT_MULT = 10.0
LOT_BASE = 0.10
LOT_MIN = 0.01
LOT_MAX = 0.20
PNL_PER_POINT = LOT_BASE * CONTRACT_MULT # 1.00 $/pt

# — Friction —————
SLIPPAGE_PTS = 2.0
MAX_SPREAD = 1.5

# — Session stricte GMT+2 —————
GMT_OFFSET = -1
SESSION_IB_START_H = 14
SESSION_IB_START_M = 0

```

```

SESSION_IB_END_H    = 14
SESSION_IB_END_M    = 30
SESSION_OPEN_H      = 15
SESSION_OPEN_M      = 30
SESSION_END_H       = 18

# — Risk Management —————
SL_ATR_MULT         = 1.5
SL_CAP_PTS          = 25.0
TP1_MIN_PTS         = 15.0
TP1_ATR_MULT        = 1.0
TP2_ATR_MULT        = 2.0
TRAIL_ATR_MULT      = 0.8
DAILY_MAX_LOSS      = -150.0
MAX_TRADES_DAY      = 3
MAX_CONSEC_SL       = 2
COOLDOWN_SECS       = 3600

# — Initial Balance & Sweep —————
IB_BUFFER_PTS       = 2.0
SWEEP_LOOKBACK      = 20
EMA_TRAIL_SPAN      = 20
ATR_PERIOD          = 14

# — [FIX-1] Limit Pullback —————
LIMIT_OFFSET_MULT   = 0.5    # entry = Close ± ATR×0.5
LIMIT_TIMEOUT_SECS  = 900    # 15 min pour fill du limit (sinon skip)

# — [FIX-3] Time-Stop —————
TIME_STOP_SECS      = 2700    # 45 min → liquidation si TP1 non atteint

# — [FIX-4] Filtre Vitesse —————
BODY_RATIO_MIN      = 0.60    # corps ≥ 60% du range (anti-doji/mèche)

print('✅ Configuration NAS100 M15 LF v2.0 OK')
print(f'  Session      : {SESSION_OPEN_H}h{SESSION_OPEN_M:02d} → {SESSION_END_H}h{00} GMT+2')
print(f'  [FIX-1] Limit: entry = Close ± ATR×{LIMIT_OFFSET_MULT} | timeout {LIMIT_TIMEOUT_SECS//60} min')
print(f'  [FIX-3] T-Stop: {TIME_STOP_SECS//60} min si TP1 non atteint')
print(f'  [FIX-4] Body : ≥ {BODY_RATIO_MIN*100:.0f}% range')
print(f'  SL          : {SL_ATR_MULT}×ATR, cap {SL_CAP_PTS} pts')
print(f'  TP1/TP2     : {TP1_ATR_MULT}/{TP2_ATR_MULT}×ATR | Slippage : {SLIPPAGE_PTS} pts')

```

- ✅ Polars 1.40.1 disponible
- ✅ Configuration NAS100 M15 LF v2.0 OK
 - Session : 15h30 → 18h00 GMT+2
 - [FIX-1] Limit: entry = Close ± ATR×0.5 | timeout 15 min
 - [FIX-3] T-Stop: 45 min si TP1 non atteint
 - [FIX-4] Body : ≥ 60% range
 - SL : 1.5×ATR, cap 25.0 pts
 - TP1/TP2 : 1.0/2.0×ATR | Slippage : 2.0 pts

```

In [2]: # =====
# CELLULE 2 – Chargement M15 & M1
# =====
def load_csv(path: Path, label: str) -> pd.DataFrame:
    df = pd.read_csv(path, parse_dates=['Time'])
    df.rename(columns={'TickVolume': 'Volume'}, inplace=True)
    df['Time'] = df['Time'] + pd.Timedelta(hours=GMT_OFFSET)
    df.set_index('Time', inplace=True)
    df.sort_index(inplace=True)
    df[['Open', 'High', 'Low', 'Close', 'Volume']] = \
        df[['Open', 'High', 'Low', 'Close', 'Volume']].astype(float)
    print(f'✅ {label} : {len(df):,} bougies | {df.index[0]} → {df.index[-1]}')
    return df

df_m15 = load_csv(PATH_M15, 'NAS100 M15')
df_m1 = load_csv(PATH_M1, 'NAS100 M1')

print('\n🔍 Format tick CSV :')
with open(PATH_TICKS, 'r') as f:
    for i, line in enumerate(f):
        print(f'  Ligne {i}: {line.strip()}')
        if i >= 3: break
print(f'\n📁 Fichier ticks : {PATH_TICKS.stat().st_size/1e9:.2f} GB')

```

✅ NAS100 M15 : 23,598 bougies | 2025-05-09 00:00:00 → 2026-05-08 22:45:00

✅ NAS100 M1 : 353,899 bougies | 2025-05-09 00:00:00 → 2026-05-08 22:59:00

🔍 Format tick CSV :

Ligne 0: Time,Bid,Ask,Last,Volume

Ligne 1: 2025.05.09 01:00:08.632,20089.55,20093.50,0.00,0

Ligne 2: 2025.05.09 01:00:09.578,20089.05,20093.00,0.00,0

Ligne 3: 2025.05.09 01:00:10.165,20090.30,20094.00,0.00,0

📁 Fichier ticks : 2.46 GB

In [3]:

```

# =====
# CELLULE 2b – Conversion CSV → Parquet (UNE SEULE FOIS)
# =====
def convert_ticks_to_parquet():
    if PATH_TICKS_PARQ.exists():
        print(f'✅ Parquet existant
({PATH_TICKS_PARQ.stat().st_size/1e9:.2f} GB) – ignoré')
        return

    print('⌚ Conversion CSV → Parquet...')
    t0 = time.time()
    with open(PATH_TICKS, 'r') as f:
        raw_cols = [c.strip().strip('<>') for c in
f.readline().strip().split(',')]
        time_col = next(c for c in raw_cols if 'time' in c.lower())
        bid_col = next(c for c in raw_cols if 'bid' in c.lower())
        ask_col = next(c for c in raw_cols if 'ask' in c.lower())
        (
            pl.scan_csv(PATH_TICKS, try_parse_dates=False,
infer_schema_length=500)
            .select([time_col, bid_col, ask_col])
            .rename({time_col: 'RawTime', bid_col: 'Bid', ask_col: 'Ask'})
            .with_columns([
                pl.col('Bid').cast(pl.Float64),
                pl.col('Ask').cast(pl.Float64),
                pl.col('RawTime')
                    .str.to_datetime(format='%Y.%m.%d %H:%M:%S%.3f',
strict=False)
                    .alias('Time'),
            ])
            .drop('RawTime')
            .sink_parquet(PATH_TICKS_PARQ, compression='snappy')
        )
    print(f'✅ Parquet créé en {time.time()-t0:.0f}s')

convert_ticks_to_parquet()
_TICKS_LAZY = pl.scan_parquet(PATH_TICKS_PARQ)
print('✅ LazyFrame Parquet NAS100 prêt')

```

- ✅ Parquet existant (0.42 GB) – ignoré
- ✅ LazyFrame Parquet NAS100 prêt

In [4]:

```

# =====
# CELLULE 3 – Indicateurs M15 [FIX-4 : body_ratio ajouté]
# =====
def compute_atr(df: pd.DataFrame, window: int = ATR_PERIOD) -> float:
    tr = pd.concat([
        df['High'] - df['Low'],
        (df['High'] - df['Close'].shift()).abs(),
        (df['Low'] - df['Close'].shift()).abs(),
    ], axis=1).max(axis=1)
    val = tr.rolling(window).mean().iloc[-1]
    return round(val, 2) if not np.isnan(val) else 0.0

def compute_ema(series: pd.Series, span: int) -> pd.Series:
    return series.ewm(span=span, adjust=False).mean()

def compute_initial_balance(df_m15: pd.DataFrame, date) -> dict:
    '''High/Low de la fenetre 14h00-14h30.'''
    day_start = pd.Timestamp(date).replace(
        hour=SESSION_IB_START_H, minute=SESSION_IB_START_M, second=0)
    day_end = pd.Timestamp(date).replace(
        hour=SESSION_IB_END_H, minute=SESSION_IB_END_M, second=0)
    ib = df_m15[(df_m15.index >= day_start) & (df_m15.index < day_end)]
    if ib.empty:
        return {'ib_high': np.nan, 'ib_low': np.nan, 'ib_valid': False}
    return {'ib_high': round(ib['High'].max(), 2),
            'ib_low' : round(ib['Low'].min(), 2),
            'ib_valid': True}

def detect_liquidity_sweep(df_m15: pd.DataFrame, ts: pd.Timestamp,
                           lookback: int = SWEEP_LOOKBACK) -> dict:
    pos = df_m15.index.searchsorted(ts)
    if pos < lookback + 2:
        return {'sweep_bull': False, 'sweep_bear': False,
                'swing_low': np.nan, 'swing_high': np.nan}
    window = df_m15.iloc[pos - lookback: pos]
    current = df_m15.iloc[pos - 1]
    swing_low = window['Low'].iloc[: -1].min()
    swing_high = window['High'].iloc[: -1].max()
    sweep_bull = (current['Low'] < swing_low
                  and current['Close'] > swing_low
                  and current['Close'] > current['Open'])
    sweep_bear = (current['High'] > swing_high
                  and current['Close'] < swing_high
                  and current['Close'] < current['Open'])
    return {'sweep_bull' : sweep_bull,
            'sweep_bear' : sweep_bear,
            'swing_low'  : round(swing_low, 2),
            'swing_high' : round(swing_high, 2)}

def detect_ib_breakout(df_m15: pd.DataFrame, ts: pd.Timestamp, ib: dict)
-> dict:
    if not ib.get('ib_valid', False):
        return {'bo_bull': False, 'bo_bear': False}
    pos = df_m15.index.searchsorted(ts)
    if pos < 1:
        return {'bo_bull': False, 'bo_bear': False}
    bar = df_m15.iloc[pos - 1]

```

```

        ib_hi = ib['ib_high'] + IB_BUFFER_PTS
        ib_lo = ib['ib_low'] - IB_BUFFER_PTS
        return {'bo_bull': (bar['Close'] > ib_hi and bar['Close'] >
bar['Open']),
                'bo_bear': (bar['Close'] < ib_lo and bar['Close'] <
bar['Open'])}

def compute_indicators_m15(df_m15: pd.DataFrame, ts: pd.Timestamp,
                           ib: dict, window: int = 100) -> dict:
    pos = df_m15.index.searchsorted(ts)
    if pos < window:
        return {}
    sub = df_m15.iloc[pos - window: pos]
    last = sub.iloc[-1]
    price = last['Close']
    atr = compute_atr(sub)
    if atr == 0:
        return {}
    ema20_s = compute_ema(sub['Close'], 20)
    ema50_s = compute_ema(sub['Close'], 50)
    ema200_s = compute_ema(sub['Close'], 200)
    vol_avg = sub['Volume'].rolling(20).mean().iloc[-1]
    vol_ratio = round(last['Volume'] / vol_avg, 2) if vol_avg > 0 else
1.0
    sweep = detect_liquidity_sweep(df_m15, ts)
    bo = detect_ib_breakout(df_m15, ts, ib)
    momentum_4 = round(sub['Close'].iloc[-1] - sub['Close'].iloc[-5], 2)
    \
        if len(sub) >= 5 else 0.0
    # [FIX-4] Filtre vélocité : ratio corps / range de la bougie signal
    bar_range = last['High'] - last['Low']
    body = abs(last['Close'] - last['Open'])
    body_ratio = round(body / bar_range, 3) if bar_range > 0 else 0.0
    return {
        'price' : round(price, 2),
        'atr' : atr,
        'ema20' : round(ema20_s.iloc[-1], 2),
        'ema50' : round(ema50_s.iloc[-1], 2),
        'ema200' : round(ema200_s.iloc[-1], 2),
        'above_200' : price > ema200_s.iloc[-1],
        'above_50' : price > ema50_s.iloc[-1],
        'ema20_slope' : round(ema20_s.diff(3).iloc[-1], 2),
        'vol_ratio' : vol_ratio,
        'momentum_4' : momentum_4,
        'sweep_bull' : sweep['sweep_bull'],
        'sweep_bear' : sweep['sweep_bear'],
        'swing_low' : sweep.get('swing_low', price - atr * 2),
        'swing_high' : sweep.get('swing_high', price + atr * 2),
        'bo_bull' : bo['bo_bull'],
        'bo_bear' : bo['bo_bear'],
        'ib_high' : ib.get('ib_high', np.nan),
        'ib_low' : ib.get('ib_low', np.nan),
        'ib_valid' : ib.get('ib_valid', False),
        'ema20_series' : ema20_s.values,
        'body_ratio' : body_ratio, # [FIX-4]
    }

```

```
print('✅ Indicateurs M15 v2.0 définis (IB + Sweep + EMA +  
body_ratio)')
```

✅ Indicateurs M15 v2.0 définis (IB + Sweep + EMA + body_ratio)

In [5]:

```

# =====
# CELLULE 4 – Direction [FIX-4], Score & Niveaux TP/SL
# =====
def determine_direction_m15(ind: dict) -> tuple:
    """
    Modèle A : Liquidity Sweep | Modèle B : IB Breakout
    [FIX-4] : body_ratio >= BODY_RATIO_MIN requis (anti-doji)
    """
    if not ind:
        return None, 'no_data'
    # [FIX-4] Filtre vélocité global
    body_ok = ind.get('body_ratio', 1.0) >= BODY_RATIO_MIN
    if not body_ok:
        return None, 'weak_body'
    # — Modèle A : Sweep —————
    if ind['sweep_bull']:
        if ind['above_200'] and ind['ema20_slope'] >= 0:
            return 'BUY', 'sweep_bull'
        elif ind['momentum_4'] > ind['atr'] * 0.3:
            return 'BUY', 'sweep_bull_momentum'
    if ind['sweep_bear']:
        if not ind['above_200'] and ind['ema20_slope'] <= 0:
            return 'SELL', 'sweep_bear'
        elif ind['momentum_4'] < -ind['atr'] * 0.3:
            return 'SELL', 'sweep_bear_momentum'
    # — Modèle B : IB Breakout —————
    if ind['ib_valid']:
        if ind['bo_bull'] and ind['above_200'] and ind['vol_ratio'] >=
1.2:
            return 'BUY', 'ib_breakout_bull'
        if ind['bo_bear'] and not ind['above_200'] and ind['vol_ratio']
>= 1.2:
            return 'SELL', 'ib_breakout_bear'
    return None, 'no_setup'

def compute_score_m15(ind: dict, direction: str) -> tuple:
    score = 0; reasons = []; is_buy = (direction == 'BUY')
    if is_buy and ind['sweep_bull']:
        score += 30;
    reasons.append('Sweep Haussier (+30)')
    elif not is_buy and ind['sweep_bear']:
        score += 30;
    reasons.append('Sweep Baissier (+30)')
    elif is_buy and ind['bo_bull']:
        score += 22;
    reasons.append('IB Breakout Bull (+22)')
    elif not is_buy and ind['bo_bear']:
        score += 22;
    reasons.append('IB Breakout Bear (+22)')
    if is_buy and ind['above_200']:
        score += 15;
    reasons.append('Above EMA200 (+15)')
    elif not is_buy and not ind['above_200']:
        score += 15;
    reasons.append('Below EMA200 (+15)')
    if is_buy and ind['above_50']:
        score += 8;
    reasons.append('Above EMA50 (+8)')
    elif not is_buy and not ind['above_50']:
        score += 8;
    reasons.append('Below EMA50 (+8)')
    slope = ind['ema20_slope']
    if is_buy and slope > 0:
        score += 10;
    reasons.append('EMA20 slope+ (+10)')
    elif not is_buy and slope < 0:
        score += 10;

```



```

reasons.append('EMA20 slope- (+10)')
    vr = ind['vol_ratio']
    if vr >= 2.0:    score += 15; reasons.append(f'VolSpike x{vr:.1f}
(+15)')
    elif vr >= 1.4: score += 10; reasons.append(f'VolHigh  x{vr:.1f}
(+10)')
    elif vr >= 1.0: score += 5; reasons.append(f'VolOK    x{vr:.1f}
(+5)')
    mom = ind['momentum_4']; atr = ind['atr']
    if is_buy and mom > atr:    score += 12;
reasons.append('Momentum+ (+12)')
    elif not is_buy and mom < -atr:    score += 12;
reasons.append('Momentum- (+12)')
    # [FIX-4] Bonus vitesse
    br = ind.get('body_ratio', 0)
    if br >= 0.80:    score += 8; reasons.append(f'VeLOCITYStrong
{br:.0%} (+8)')
    elif br >= 0.60: score += 4; reasons.append(f'VeLOCITYOK {br:.0%}
(+4)')
    return min(score, 100), reasons

def calculate_levels_m15(direction: str, entry: float, atr: float,
                        swing_high: float, swing_low: float) -> dict:
    raw_risk = atr * SL_ATR_MULT
    if direction == 'BUY':
        sl = round(min(swing_low - IB_BUFFER_PTS, entry - raw_risk),
2)

        sl = round(max(sl, entry - SL_CAP_PTS), 2)
        risk = entry - sl
        tp1d = max(TP1_MIN_PTS, TP1_ATR_MULT * atr)
        return {'sl': sl, 'tp1': round(entry + tp1d, 2),
                'tp2': round(entry + TP2_ATR_MULT * atr, 2),
                'tp3': None, 'risk': round(risk, 2)}
    else:
        sl = round(max(swing_high + IB_BUFFER_PTS, entry + raw_risk),
2)

        sl = round(min(sl, entry + SL_CAP_PTS), 2)
        risk = sl - entry
        tp1d = max(TP1_MIN_PTS, TP1_ATR_MULT * atr)
        return {'sl': sl, 'tp1': round(entry - tp1d, 2),
                'tp2': round(entry - TP2_ATR_MULT * atr, 2),
                'tp3': None, 'risk': round(risk, 2)}

def compute_lot_dynamic(score: int, atr: float, median_atr: float) ->
float:
    mult = 1.0
    if score >= 80:    mult *= 1.2
    elif score <= 50: mult *= 0.8
    if median_atr > 0 and atr > 1.6 * median_atr:
        mult *= 0.7
    return max(LOT_MIN, min(LOT_MAX, round(LOT_BASE * mult, 2)))

print('✅ Direction [FIX-4], Score, Niveaux v2.0 définis')

```

✅ Direction [FIX-4], Score, Niveaux v2.0 définis

```

In [6]: # =====
# CELLULE 5 – Fetch Ticks + Simulateur v2.0
# [FIX-1] Limit Pullback [FIX-2] BE inconditionnel
# [FIX-3] Time-Stop 45min [FIX-6] Timestamps dans cache
# =====
def _fetch_ticks_numpy(signal_time_pc: pd.Timestamp,
                      max_hours: int = 4) -> tuple:
    '''Retourne (ts_epoch_f64, bids, asks) ou 3 arrays vides.'''
    broker_start = signal_time_pc + timedelta(hours=-GMT_OFFSET)
    broker_end = broker_start + timedelta(hours=max_hours)
    df = (_TICKS_LAZY
          .filter((pl.col('Time') >= broker_start) &
                  (pl.col('Time') <= broker_end))
          .select(['Time', 'Bid', 'Ask'])
          .collect(streaming=True))
    if df.is_empty():
        return np.array([]), np.array([]), np.array([])
    # Timestamps en secondes (float64) pour arithmetic rapide
    ts_arr = (df['Time'].cast(pl.Int64).to_numpy() /
1e9).astype(np.float64)
    return ts_arr, df['Bid'].to_numpy(), df['Ask'].to_numpy()

def simulate_trade_ticks(direction: str, entry_signal: float,
                        levels: dict, ts_arr: np.ndarray,
                        bids: np.ndarray, asks: np.ndarray,
                        atr: float, ema20_trail=None,
                        slippage: float = SLIPPAGE_PTS) -> list:
    ...

    Simulateur Tick v2.0 – 3 positions (P1/P2/P3)
    [FIX-1] Limit Pullback : attend entry_signal +/- ATR*0.5
    [FIX-2] BE inconditionnel sur P2/P3 dès TP1 touché
    [FIX-3] Time-Stop 45min si TP1 non atteint
    ...

    if len(bids) == 0:
        return []
    if asks[0] - bids[0] > MAX_SPREAD:
        return []

    # — [FIX-1] Calcul du prix limite —————
    if direction == 'BUY':
        limit_px = round(entry_signal - atr * LIMIT_OFFSET_MULT, 2)
        exec_check = asks # BUY : on surveille les asks
        exec_px = bids # PnL calculé sur bid
    else:
        limit_px = round(entry_signal + atr * LIMIT_OFFSET_MULT, 2)
        exec_check = bids # SELL : on surveille les bids
        exec_px = asks

    # Chercher le premier tick où le limit est touché (fenêtre 15 min)
    limit_idx = None
    limit_cutoff = ts_arr[0] + LIMIT_TIMEOUT_SECS
    for k in range(len(exec_check)):
        if ts_arr[k] > limit_cutoff:
            break
        if direction == 'BUY' and exec_check[k] <= limit_px:
            limit_idx = k; break

```

```

elif direction == 'SELL' and exec_check[k] >= limit_px:
    limit_idx = k; break

if limit_idx is None:
    return [] # Limit non touchée dans la fenêtre → pas de trade

# Entry réelle = limit_px ± slippage
entry = round(limit_px + slippage if direction == 'BUY'
              else limit_px - slippage, 2)
delta = entry - entry_signal # pour ajuster TP/SL

sl_init = round(levels['sl'] + delta, 2)
tp1      = round(levels['tp1'] + delta, 2)
tp2      = round(levels['tp2'] + delta, 2)

positions = [
    {'id':'P1','tp':tp1, 'sl':sl_init,'result':None,'exit_px':None,
     'pnl':0.,'closed':False,'entry_real':entry,
     'slippage':round(slippage, 4),'mfe':0.,'mae':0.},
    {'id':'P2','tp':tp2, 'sl':sl_init,'result':None,'exit_px':None,
     'pnl':0.,'closed':False,'entry_real':entry,
     'slippage':round(slippage, 4),'mfe':0.,'mae':0.},
    {'id':'P3','tp':None,'sl':sl_init,'result':None,'exit_px':None,
     'pnl':0.,'closed':False,'entry_real':entry,
     'slippage':round(slippage,
4),'mfe':0.,'mae':0.,'trailing':False},
]

tp1_hit = tp2_hit = False
start_ts = ts_arr[limit_idx] # référence Time-Stop

for k in range(limit_idx, len(exec_px)):
    if all(p['closed'] for p in positions):
        break

    px      = exec_px[k]
    elapsed = ts_arr[k] - start_ts

    # — [FIX-3] Time-Stop 45 min —————
    if elapsed > TIME_STOP_SECS and not tp1_hit:
        for p in positions:
            if not p['closed']:
                pts = (px - entry) if direction == 'BUY' else (entry
- px)

                p['result'] = 'TIME_STOP'
                p['exit_px'] = round(px, 2)
                p['pnl']    = round(pts * PNL_PER_POINT, 2)
                p['closed'] = True
            break

    # MFE / MAE
    for p in positions:
        if not p['closed']:
            exc = (px - entry) if direction == 'BUY' else (entry -
px)

            p['mfe'] = max(p['mfe'], exc)
            p['mae'] = min(p['mae'], exc)

```

```

# Trailing P3
p3 = positions[2]
if not p3['closed'] and p3['trailing']:
    if ema20_trail is not None and len(ema20_trail) > 0:
        trail_level = ema20_trail[min(k - limit_idx,
                                      len(ema20_trail) - 1)]
    else:
        trail_level = (px - atr * TRAIL_ATR_MULT if direction ==
'BUY'
                      else px + atr * TRAIL_ATR_MULT)
    new_sl = round(trail_level - atr * 0.1 if direction == 'BUY'
                  else trail_level + atr * 0.1, 2)
    if direction == 'BUY' and new_sl > p3['sl']: p3['sl'] =
new_sl
    if direction == 'SELL' and new_sl < p3['sl']: p3['sl'] =
new_sl

for p in positions:
    if p['closed']:
        continue

# P3 en mode trailing : seul le SL compte
if p['id'] == 'P3' and p['trailing']:
    sl_h = (px <= p['sl']) if direction == 'BUY' else (px >=
p['sl'])
    if sl_h:
        p['closed'] = True
        p['exit_px'] = p['sl']
        rpts = ((p['sl'] - entry) if direction == 'BUY'
                else (entry - p['sl']))
        p['result'] = 'TRAIL_CLOSE' if rpts > 0 else 'SL'
        p['pnl'] = round(rpts * PNL_PER_POINT, 2)
        continue

# [FIX-5] TP check sécurisé (tp_v peut être None pour P3)
tp_v = p['tp']
th = ((px >= tp_v) if direction == 'BUY' else (px <= tp_v))
\
    if tp_v is not None else False
sh = (px <= p['sl']) if direction == 'BUY' else (px >=
p['sl'])

# Résolution conflits TP/SL sur même tick
if th and sh and tp_v is not None:
    th = abs(tp_v - entry) < abs(p['sl'] - entry)
    sh = not th

if th:
    p['closed'] = True
    p['exit_px'] = tp_v
    pts = (tp_v - entry) if direction == 'BUY' else (entry -
tp_v)

    p['result'] = p['id'].replace('P', 'TP')
    p['pnl'] = round(pts * PNL_PER_POINT, 2)

    if p['id'] == 'P1' and not tp1_hit:

```

```

tp1_hit = True
# — [FIX-2] BREAKEVEN INCONDITIONNEL —————
for o in positions:
    if not o['closed']:
        o['sl'] = entry    # ← INCONDITIONNEL (aucun
test)

    if p['id'] == 'P2' and not tp2_hit and tp1_hit:
        tp2_hit = True
        if not positions[2]['closed']:
            positions[2]['trailing'] = True
            # SL initial trailing = 25% entre TP1 et TP2
            positions[2]['sl'] = (
                round(tp1 + (tp2 - tp1) * 0.25, 2)
                if direction == 'BUY'
                else round(tp1 - (tp1 - tp2) * 0.25, 2)
            )

        elif sh:
            p['closed'] = True
            p['exit_px'] = p['sl']
            pts = abs(entry - p['sl'])
            is_be = abs(p['sl'] - entry) < 0.5
            p['result'] = 'BE' if is_be else 'SL'
            p['pnl'] = 0. if is_be else round(-pts *
PNL_PER_POINT, 2)

        # Force-close positions encore ouvertes (fin données)
        last_px = exec_px[-1]
        for p in positions:
            if not p['closed']:
                pts = (last_px - entry) if direction == 'BUY' else (entry -
last_px)
                p['result'] = 'OPEN_END'
                p['exit_px'] = last_px
                p['pnl'] = round(pts * PNL_PER_POINT, 2)
                p['closed'] = True

        return positions

print('✅ Fetch ticks v2.0 + Simulateur [FIX-1,2,3,5] défini')

```

✅ Fetch ticks v2.0 + Simulateur [FIX-1,2,3,5] défini

In [7]:

```

# =====
# CELLULE 6 – Pré-calcul indicateurs M15 + cache IB
# =====
def build_ib_cache(df_m15: pd.DataFrame) -> dict:
    dates = df_m15.index.normalize().unique()
    cache = {d.date(): compute_initial_balance(df_m15, d.date()) for d
in dates}
    valid = sum(1 for v in cache.values() if v['ib_valid'])
    print(f'✅ IB calculée pour {valid}/{len(cache)} jours')
    return cache

def precompute_all_indicators(df_m15: pd.DataFrame,
                             ib_cache: dict, window: int = 100) ->
OrderedDict:
    if CACHE_IND.exists():
        print('📁 Cache v2.0 détecté – chargement...')
        with open(CACHE_IND, 'rb') as f:
            result = pickle.load(f)
            print(f'✅ {len(result):,} timestamps chargés')
            return result
    times = df_m15.index.tolist()
    precomp = OrderedDict()
    t0 = time.time()
    with tqdm(total=len(df_m15)-window, desc='Pré-calcul M15 v2.0',
              unit='bougies', ncols=80) as pbar:
        for i in range(window, len(df_m15)):
            ts = times[i]
            ib = ib_cache.get(ts.date(), {'ib_valid': False})
            try:
                ind = compute_indicators_m15(df_m15, ts, ib, window)
                if ind: precomp[ts] = ind
            except Exception:
                pass
            pbar.update(1)
    print(f'\n✅ {len(precomp):,} timestamps en {time.time()-t0:.1f}s')
    with open(CACHE_IND, 'wb') as f:
        pickle.dump(precomp, f)
    return precomp

print('⌚ Calcul IB cache...')
ib_cache = build_ib_cache(df_m15)

if 'precomp_indicators' in globals() and precomp_indicators:
    print('✅ Indicateurs déjà en RAM – ignoré.')
else:
    print('⌚ Pré-calcul indicateurs M15 v2.0...')
    precomp_indicators = precompute_all_indicators(df_m15, ib_cache)

```

⌚ Calcul IB cache...

✅ IB calculée pour 258/259 jours

⌚ Pré-calcul indicateurs M15 v2.0...

Pré-calcul M15 v2.0: 0%

| 0/23498

[00:00<?, ?bougies/s]

✅ 23,498 timestamps en 75.8s

```

In [8]: # =====
# CELLULE 6b – Cache ticks v2.0 [FIX-6 : timestamps inclus]
# exec_time = ts + 15 min (clôture bougie M15)
# Cache = dict[ts] → (ts_arr, bids, asks)
# =====
def _collect_signal_times(precomp: OrderedDict, min_score: int = 35) -> list:
    times = []; last_ts = pd.Timestamp('1970-01-01')
    cur_date = None; trades_today = 0
    for ts, ind in precomp.items():
        day = ts.date()
        if day != cur_date:
            cur_date = day; trades_today = 0
        if trades_today >= MAX_TRADES_DAY: continue
        h, mn = ts.hour, ts.minute
        in_session = ((h > SESSION_OPEN_H or
                        (h == SESSION_OPEN_H and mn >= SESSION_OPEN_M))
                       and h < SESSION_END_H)
        if not in_session: continue
        if (ts - last_ts).total_seconds() < COOLDOWN_SECS: continue
        direction, _ = determine_direction_m15(ind)
        if direction is None: continue
        score, _ = compute_score_m15(ind, direction)
        if score < min_score: continue
        times.append(ts); last_ts = ts; trades_today += 1
    return times

def build_tick_cache(signal_times: list, max_hours: int = 4) -> dict:
    '''Cache → dict[ts] = (ts_arr, bids, asks) [FIX-6]'''
    cache = {}
    print(f'📦 Chargement de {len(signal_times)} fenêtres de ticks...')
    t0 = time.time()
    for ts in tqdm(signal_times, desc='Cache ticks v2.0', unit='trade',
ncols=70):
        exec_time = ts + pd.Timedelta(minutes=15)
        ts_arr, bids, asks = _fetch_ticks_numpy(exec_time, max_hours)
        if len(bids) > 0:
            cache[ts] = (ts_arr, bids, asks)
        elapsed = time.time() - t0
        total_mb = sum(t.nbytes + b.nbytes + a.nbytes
                        for t, b, a in cache.values()) / 1e6
        print(f'\n✅ Cache : {len(cache)} trades | {total_mb:.1f} MB |
{elapsed:.0f}s')
    return cache

print('🔍 Collecte timestamps candidats (score_min=35)...')
all_signal_times = _collect_signal_times(precomp_indicators,
min_score=35)
print(f'→ {len(all_signal_times)} signaux candidats\n')
print('⌚ Construction cache ticks v2.0...')
TICK_CACHE = build_tick_cache(all_signal_times)

```

🔍 Collecte timestamps candidats (score_min=35)...
→ 284 signaux candidats

⌚ Construction cache ticks v2.0...
 📦 Chargement de 284 fenêtres de ticks...

Cache ticks v2.0: 0%| | 0/284 [00:00<?, ?
 trade/s]

✅ Cache : 284 trades | 379.5 MB | 2s

In [9]:

```
# =====
# CELLULE 7 – ATR médiane 30j
# =====
def compute_median_atr_30d(precomp: OrderedDict) -> pd.Series:
    records = [(ts, ind['atr']) for ts, ind in precomp.items()]
    s = pd.Series([r[1] for r in records],
                  index=pd.DatetimeIndex([r[0] for r in records]))
    return s.rolling('30D').median()

print('⌚ Calcul ATR médiane 30j...')
median_atr_series = compute_median_atr_30d(precomp_indicators)
print(f'✅ ATR médiane | min={median_atr_series.min():.1f} '
      f'max={median_atr_series.max():.1f} pts')
```

⌚ Calcul ATR médiane 30j...
 ✅ ATR médiane | min=17.6 max=55.5 pts

In [10]:

```

# =====
# CELLULE 8 – Moteur de backtest M15 LF v2.0
# =====
def run_backtest_m15(precomp: OrderedDict, tick_cache: dict,
                    median_atr: pd.Series, score_min: int = 45,
                    label: str = 'NAS100 M15 LF v2.0') -> tuple:
    trades_log = []
    daily_pnl = 0.; trades_today = 0; consec_sl = 0
    last_sl_ts = pd.Timestamp('1970-01-01')
    last_sig_ts = pd.Timestamp('1970-01-01')
    cur_date = None; daily_stopped = False
    fs = {k: 0 for k in
['no_session', 'no_setup', 'score_low', 'weak_body',
    'limit_miss', 'spread_rej', 'cooldown',
    'consec_sl', 'daily_stop', 'freq_limit']}

    for ts, ind in tqdm(precomp.items(), desc=f'🚀 {label}',
                        unit='bougies', ncol=82):
        day = ts.date()
        if day != cur_date:
            cur_date = day; daily_pnl = 0.; trades_today = 0
            consec_sl = 0; daily_stopped = False

            if daily_stopped:
                fs['daily_stop'] += 1;
            continue
            if daily_pnl < DAILY_MAX_LOSS:
                daily_stopped = True;
            fs['daily_stop'] += 1; continue
            if consec_sl >= MAX_CONSEC_SL:
                fs['consec_sl'] += 1;
            continue
            if trades_today >= MAX_TRADES_DAY:
                fs['freq_limit'] += 1;
            continue

            h, mn = ts.hour, ts.minute
            in_session = ((h > SESSION_OPEN_H or
                (h == SESSION_OPEN_H and mn >= SESSION_OPEN_M))
                and h < SESSION_END_H)
            if not in_session:
                fs['no_session'] += 1;
            continue

            cooldown_eff = COOLDOWN_SECS * (1.5 ** consec_sl)
            if (ts - last_sig_ts).total_seconds() < cooldown_eff:
                fs['cooldown'] += 1;
            continue

            direction, reject = determine_direction_m15(ind)
            if direction is None:
                if reject == 'weak_body':
                    fs['weak_body'] += 1
                else:
                    fs['no_setup'] += 1
                continue

            score, _ = compute_score_m15(ind, direction)
            if score < score_min:
                fs['score_low'] += 1;
            continue

            if ts not in tick_cache:
                fs['limit_miss'] += 1;
            continue

```

```

entry_signal = ind['price']; atr = ind['atr']
levels = calculate_levels_m15(direction, entry_signal, atr,
                             ind['swing_high'],
ind['swing_low'])
med_atr = median_atr.get(ts, atr)
lot = compute_lot_dynamic(score, atr, med_atr)
pnl_pt = round(lot * CONTRACT_MULT, 4)

ts_arr, bids, asks = tick_cache[ts] # [FIX-6]
positions = simulate_trade_ticks(
    direction, entry_signal, levels,
    ts_arr=ts_arr, bids=bids, asks=asks, atr=atr,
    ema20_trail=ind.get('ema20_series'),
    slippage=SLIPPAGE_PTS)

if not positions:
    fs['limit_miss'] += 1; continue # limit non touchée

# Spread reject comptabilisé dans simulate (retour [])
for p in positions:
    if p['pnl'] != 0.:
        p['pnl'] = round(p['pnl'] * (pnl_pt / PNL_PER_POINT), 2)

seq_pnl = sum(p['pnl'] for p in positions)
daily_pnl = round(daily_pnl + seq_pnl, 2)
trades_today += 1; last_sig_ts = ts

if any(p['result'] == 'SL' for p in positions):
    consec_sl += 1; last_sl_ts = ts
else:
    consec_sl = 0

entry_real = positions[0].get('entry_real', entry_signal)
slip_val = positions[0].get('slippage', SLIPPAGE_PTS)
limit_disc = round(entry_signal - entry_real, 2) \
    if direction == 'BUY' \
    else round(entry_real - entry_signal, 2)

for p in positions:
    trades_log.append({
        'open_time' : ts,
        'day' : day,
        'direction' : direction,
        'signal' : reject,
        'entry_signal': entry_signal,
        'entry_real' : entry_real,
        'slippage' : slip_val,
        'limit_disc' : limit_disc,
        'atr' : atr,
        'body_ratio' : ind.get('body_ratio', 0.),
        'lot' : lot,
        'pnl_per_pt' : pnl_pt,
        'score' : score,
        'sl' : levels['sl'],
        'tp1' : levels['tp1'],
        'tp2' : levels['tp2'],
        'risk_pts' : levels['risk'],

```

```

        'position'      : p['id'],
        'result'       : p['result'],
        'exit_px'      : p['exit_px'],
        'pnl'          : p['pnl'],
        'mfe'          : p.get('mfe', 0.),
        'mae'          : p.get('mae', 0.),
        'daily_pnl'    : daily_pnl,
        'hour'         : h,
        'ib_high'      : ind.get('ib_high', np.nan),
        'ib_low'       : ind.get('ib_low', np.nan),
        'vol_ratio'    : ind['vol_ratio'],
        'consec_sl'    : consec_sl,
    })

    return pd.DataFrame(trades_log), fs

print('✅ Moteur de backtest NAS100 M15 LF v2.0 défini')

```

✅ Moteur de backtest NAS100 M15 LF v2.0 défini

In [11]:

```

# =====
# CELLULE 9 – Lancement backtest v2.0
# =====
print('🚀 Lancement backtest M15 LF v2.0...')
t0 = time.time()
df_trades, fs_tick = run_backtest_m15(
    precomp_indicators, TICK_CACHE, median_atr_series, score_min=45
)
n_seq = len(df_trades) // 3 if not df_trades.empty else 0
print(f'✅ Terminé en {time.time()-t0:.1f}s | {n_seq} séquences')

```

🚀 Lancement backtest M15 LF v2.0...

🚀 NAS100 M15 LF v2.0: 0% | 0/23498
[00:00<?, ?bougies/s]

✅ Terminé en 6.5s | 185 séquences

In [12]:

```

# =====
# CELLULE 10 – Statistiques NAS100 M15 LF v2.0
# =====
def compute_stats(df: pd.DataFrame, fs: dict, label: str) -> dict:
    if df.empty:
        print(f'⚠️ [{label}] Aucun trade.');
```

```

1)
2)

```

```

        return {}
    seq  = df.groupby('open_time')['pnl'].sum()
    net   = round(df['pnl'].sum(), 2)
    gw    = df[df['pnl'] > 0]['pnl'].sum()
    gl    = abs(df[df['pnl'] < 0]['pnl'].sum())
    pf    = round(gw / gl, 2) if gl > 0 else float('inf')
    n_seq = len(seq)
    wr    = round((seq > 0).sum() / n_seq * 100, 1) if n_seq > 0 else 0
    equity = seq.cumsum()
    dd    = round((equity - equity.cummax()).min(), 2)
    n_days = max((df['open_time'].max() - df['open_time'].min()).days,
    calmar = round(net * 365 / n_days / abs(dd), 2) if dd < 0 else
float('inf')
    avg_win = df[df['pnl'] > 0]['pnl'].mean() if (df['pnl'] > 0).any()
else 0
    avg_loss = df[df['pnl'] < 0]['pnl'].mean() if (df['pnl'] < 0).any()
else 0
    expectancy = round(wr / 100 * avg_win + (1 - wr / 100) * avg_loss,
    sep = '=' * 64
    print(f'\n{sep}')
    print(f' [{label}]')
    print(f' Période      : {df["open_time"].min().date()} → '
          f'{df["open_time"].max().date()}')
    print(f' Séquences      : {n_seq}')
    print(f' Net Profit       : {net:+.2f} $')
    print(f' Profit Factor    : {pf}')
    print(f' Max Drawdown     : {dd:.2f} $')
    print(f' Calmar Ratio      : {calmar}')
    print(f' Winrate           : {wr} %')
    print(f' Espérance/trd     : {expectancy:+.2f} $')
    print(f' Meilleure seq     : {seq.max():+.2f} $')
    print(f' Pire seq          : {seq.min():+.2f} $')

    print(f'\n Résultats par palier :')
    for tp in
['TP1', 'TP2', 'TRAIL_CLOSE', 'TIME_STOP', 'SL', 'BE', 'OPEN_END']:
        n = (df['result'] == tp).sum()
        pn = df[df['result'] == tp]['pnl'].sum()
        if n > 0:
            print(f' {tp:12s}: {n:4d} | PnL : {pn:+.2f} $')

    print(f'\n Friction & Exécution :')
    slip_mean = df.groupby('open_time')['slippage'].first().mean()
    disc_mean = df.groupby('open_time')['limit_disc'].first().mean() \
        if 'limit_disc' in df.columns else 0.
    print(f' Slippage fixe : {slip_mean:.2f} pts')
    print(f' Discount limit: {disc_mean:+.2f} pts (économie vs
marché)')
    print(f' Spread rej    : {fs.get("spread_rej", 0)}')
```

```

print(f'    Limit miss    : {fs.get("limit_miss", 0)}')
print(f'    Weak body rej : {fs.get("weak_body", 0)}')

print(f'\n Sources de signal :')
if 'signal' in df.columns:
    for sig, cnt in df.groupby('open_time')['signal'].first() \
        .value_counts().items():
        print(f'    {sig:<30}: {cnt}')
print(f'{sep}')

return {'net': net, 'pf': pf, 'dd': dd, 'wr': wr,
        'calmar': calmar, 'n_seq': n_seq,
        'equity': equity, 'expectancy': expectancy}

stats_tick = compute_stats(df_trades, fs_tick, 'NAS100 M15 LF v2.0')

```

```

=====
[NAS100 M15 LF v2.0]
Période      : 2025-05-16 → 2026-05-07
Séquences    : 185
Net Profit   : -4444.05 $
Profit Factor : 0.54
Max Drawdown : -5536.42 $
Calmar Ratio  : -0.83
Winrate      : 27.0 %
Espérance/trd : -2.65 $
Meilleure seq : +332.55 $
Pire seq     : -90.00 $

Résultats par palier :
TP1          : 49 | PnL : +2018.01 $
TP2          : 18 | PnL : +1430.96 $
TRAIL_CLOSE  : 11 | PnL : +580.04 $
SL           : 405 | PnL : -9750.00 $
BE           : 54 | PnL : +0.00 $
OPEN_END     : 18 | PnL : +1276.94 $

Friction & Exécution :
Slippage fixe : 2.00 pts
Discount limit: +21.53 pts (économie vs marché)
Spread rej    : 0
Limit miss    : 144
Weak body rej : 1221

Sources de signal :
ib_breakout_bull      : 90
ib_breakout_bear      : 88
sweep_bear_momentum   : 4
sweep_bull_momentum   : 3
=====

```

In [13]:

```

# =====
# CELLULE 11 – Graphiques analytiques v2.0
# =====
if not df_trades.empty and stats_tick:
    pal = {'TP1': '#00ccff', 'TP2': '#0099cc', 'TRAIL_CLOSE': '#006699',
           'TIME_STOP': '#ff9900', 'SL': '#ff4466', 'BE': '#ffdd44',
           'OPEN_END': '#888888'}
    fig = plt.figure(figsize=(24, 20))
    fig.suptitle('NAS100 M15 LF v2.0 – Limit Pullback + BE Fix + Time-
Stop',
                fontsize=14, fontweight='bold', color='white', y=0.99)
    eq = stats_tick['equity']

    # (1) Equity Curve
    ax1 = fig.add_subplot(3, 3, (1, 3))
    ax1.fill_between(eq.index, eq.values, alpha=0.15, color='#00ccff')
    ax1.plot(eq.index, eq.values, lw=2, color='#00ccff',
            label=f'Net: {eq.iloc[-1]:+.0f}$')
    ax1.axhline(0, color='white', lw=0.5, linestyle='--', alpha=0.4)
    ax1.set_title('📈 Equity Curve NAS100 M15 LF v2.0',
                fontsize=12, color='#00d4ff')
    ax1.set_ylabel('PnL cumulé ($)', color='white')
    ax1.tick_params(colors='white'); ax1.grid(alpha=0.15);
ax1.legend(fontsize=10)

    # (2) PnL journalier
    ax2 = fig.add_subplot(3, 3, 4)
    daily = df_trades.groupby('day')['pnl'].sum()
    bar_c = ['#00ccff' if v >= 0 else '#ff4466' for v in daily.values]
    ax2.bar(range(len(daily)), daily.values, color=bar_c, alpha=0.85)
    ax2.axhline(0, color='white', lw=0.5)
    ax2.axhline(DAILY_MAX_LOSS, color='#ff8800', lw=1.2, linestyle='--',
            label=f'Daily Stop {DAILY_MAX_LOSS}$')
    ax2.set_title('📊 PnL Journalier', fontsize=11, color='#00d4ff')
    ax2.set_ylabel('PnL ($)', color='white')
    ax2.tick_params(colors='white'); ax2.grid(alpha=0.15, axis='y')
    ax2.legend(fontsize=8)

    # (3) Répartition résultats
    ax3 = fig.add_subplot(3, 3, 5)
    rc = df_trades['result'].value_counts()
    ax3.pie(rc.values, labels=rc.index, autopct='%1.1f%%',
            colors=[pal.get(r, '#aaaaaa') for r in rc.index],
            textprops={'color': 'white', 'fontsize': 9})
    ax3.set_title('🎯 Résultats Positions', fontsize=11,
color='#00d4ff')

    # (4) MFE vs MAE
    ax4 = fig.add_subplot(3, 3, 6)
    if 'mfe' in df_trades.columns:
        df_p1 = df_trades[df_trades['position'] == 'P1'].copy()
        for res in df_p1['result'].unique():
            sub = df_p1[df_p1['result'] == res]
            ax4.scatter(sub['mae'], sub['mfe'], c=pal.get(res,
'#aaaaaa'),
                        label=res, alpha=0.75, s=60)
        ax4.axhline(0, color='white', lw=0.5, alpha=0.3)

```

```

ax4.axvline(0, color='white', lw=0.5, alpha=0.3)
ax4.set_title('MFE vs MAE (P1)', fontsize=11, color='#00d4ff')
ax4.set_xlabel('MAE (pts)', color='white')
ax4.set_ylabel('MFE (pts)', color='white')
ax4.tick_params(colors='white'); ax4.grid(alpha=0.15);
ax4.legend(fontsize=7)

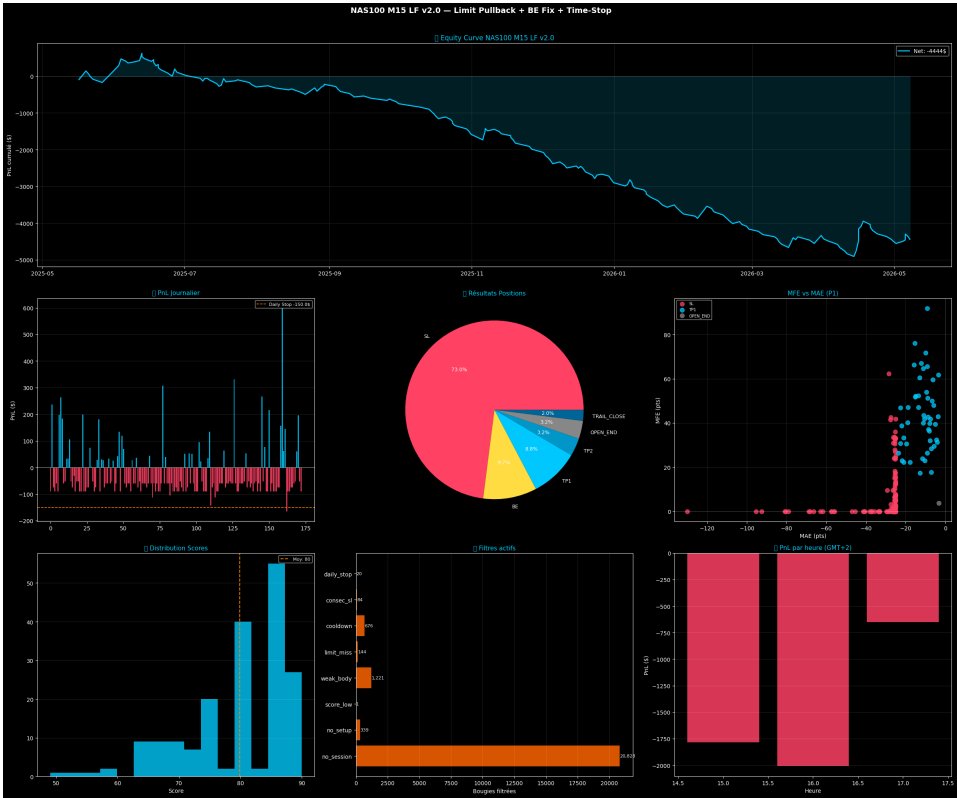
# (5) Distribution scores
ax5 = fig.add_subplot(3, 3, 7)
scores = df_trades.groupby('open_time')['score'].first()
ax5.hist(scores.values, bins=15, color='#00ccff', alpha=0.8,
edgecolor='none')
ax5.axvline(scores.mean(), color='#ff8800', lw=1.5, linestyle='--',
label=f'Moy: {scores.mean():.0f}')
ax5.set_title('📊 Distribution Scores', fontsize=11,
color='#00d4ff')
ax5.set_xlabel('Score', color='white')
ax5.tick_params(colors='white'); ax5.grid(alpha=0.15, axis='y')
ax5.legend(fontsize=8)

# (6) Filtres actifs
ax6 = fig.add_subplot(3, 3, 8)
lf = [k for k, v in fs_tick.items() if v > 0]
vf = [fs_tick[k] for k in lf]
bars6 = ax6.barh(lf, vf, color='#ff6600', alpha=0.85)
ax6.set_title('📊 Filtres actifs', fontsize=11, color='#00d4ff')
ax6.set_xlabel('Bougies filtrées', color='white')
ax6.tick_params(colors='white'); ax6.grid(alpha=0.15, axis='x')
for bar, val in zip(bars6, vf):
    ax6.text(bar.get_width() + 5, bar.get_y() + bar.get_height() /
2,
f'{val:,.}', va='center', color='white', fontsize=8)

# (7) PnL par heure
ax7 = fig.add_subplot(3, 3, 9)
pnl_hour = df_trades.groupby('hour')['pnl'].sum()
c7 = ['#00ccff' if v >= 0 else '#ff4466' for v in pnl_hour.values]
ax7.bar(pnl_hour.index, pnl_hour.values, color=c7, alpha=0.85)
ax7.axhline(0, color='white', lw=0.5)
ax7.set_title('🕒 PnL par heure (GMT+2)', fontsize=11,
color='#00d4ff')
ax7.set_xlabel('Heure', color='white')
ax7.set_ylabel('PnL ($)', color='white')
ax7.tick_params(colors='white'); ax7.grid(alpha=0.15, axis='y')

plt.tight_layout(rect=[0, 0, 1, 0.98])
plt.savefig('nas100_m15_lf_v2_results.png', dpi=150,
bbox_inches='tight', facecolor='#d0d0d0',
edgecolor='none')
plt.show()
print('✅ Graphique → nas100_m15_lf_v2_results.png')

```

Graphique → nas100_m15_lf_v2_results.png

In [14]:

```

# =====
# CELLULE 12 – MFE / MAE
# =====
if not df_trades.empty and 'mfe' in df_trades.columns:
    pal_res = {'TP1': '#00ccff', 'TP2': '#0099cc', 'TRAIL_CLOSE': '#006699',
               'TIME_STOP': '#ff9900', 'SL': '#ff4466', 'BE': '#ffdd44',
               'OPEN_END': '#888888'}
    df_p1 = df_trades[df_trades['position'] == 'P1'].copy()
    fig2, axes2 = plt.subplots(1, 2, figsize=(20, 7))
    fig2.suptitle('NAS100 M15 LF v2.0 – Analyse MFE / MAE',
                  fontsize=13, fontweight='bold', color='white')

    ax_s = axes2[0]
    for res in df_p1['result'].unique():
        sub = df_p1[df_p1['result'] == res]
        ax_s.scatter(sub['mae'], sub['mfe'], c=pal_res.get(res,
        'aaaaaa'),
                    label=res, alpha=0.75, s=55, edgecolors='none')
    ax_s.axhline(0, color='white', lw=0.5, alpha=0.4)
    ax_s.axvline(0, color='white', lw=0.5, alpha=0.4)
    ax_s.set_xlabel('MAE (pts)', color='white')
    ax_s.set_ylabel('MFE (pts)', color='white')
    ax_s.set_title('Scatter MFE vs MAE (P1)', color='#00d4ff')
    ax_s.tick_params(colors='white'); ax_s.grid(alpha=0.15)
    ax_s.legend(fontsize=8)

    ax_b = axes2[1]
    result_order = ['TP1', 'TP2', 'TRAIL_CLOSE', 'TIME_STOP', 'SL',
    'BE']
    data_box = [df_p1[df_p1['result'] == r]['mae'].dropna().values
                 for r in result_order if (df_p1['result'] == r).any()]
    labels_box = [r for r in result_order if (df_p1['result'] ==
    r).any()]
    bp = ax_b.boxplot(data_box, labels=labels_box, patch_artist=True,
                      medianprops=dict(color='white', lw=2))
    for patch, lbl in zip(bp['boxes'], labels_box):
        patch.set_facecolor(pal_res.get(lbl, 'aaaaaa'))
        patch.set_alpha(0.75)
    ax_b.axhline(0, color='white', lw=0.5, linestyle='--', alpha=0.4)
    ax_b.set_xlabel('Résultat P1', color='white')
    ax_b.set_ylabel('MAE (pts)', color='white')
    ax_b.set_title('Distribution MAE par Résultat', color='#00d4ff')
    ax_b.tick_params(colors='white'); ax_b.grid(alpha=0.15, axis='y')

    print(f'\n🌈 MFE/MAE (P1) :')
    print(f' MFE moyen      : {df_p1["mfe"].mean():+.1f} pts')
    print(f' MAE moyen        : {df_p1["mae"].mean():+.1f} pts')
    for r in ['SL', 'TP1', 'TP2', 'TIME_STOP', 'BE']:
        if (df_p1['result'] == r).any():
            print(f' MAE ({r:10s}): '
                  f'{df_p1[df_p1["result"]==r]["mae"].mean():+.1f} pts')

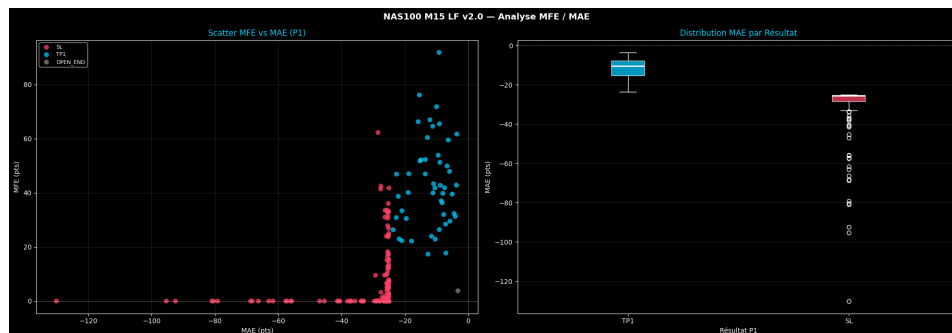
    plt.tight_layout()
    plt.savefig('nas100_m15_mfe_mae_v2.png', dpi=130,
                bbox_inches='tight', facecolor='#0d0d0d')

```

```
plt.show()  
print('✅ MFE/MAE → nas100_m15_mfe_mae_v2.png')
```

📊 MFE/MAE (P1) :

MFE moyen : +16.7 pts
MAE moyen : -26.9 pts
MAE (SL) : -32.5 pts
MAE (TP1) : -11.9 pts



✅ MFE/MAE → nas100_m15_mfe_mae_v2.png

In [15]:

```

# =====
# CELLULE 13 – Sensibilité parallèle v2.0
# [FIX-5] : tp_v is not None avant px>=tp_v dans _sim worker
# =====
N_CORES = multiprocessing.cpu_count()
print(f'🖨️ CPU : {N_CORES} cœurs disponibles')

def _sensitivity_worker(args: tuple) -> dict:
    (precomp_items, tick_cache_local, sm, slm,
     pnl_pt_base, slip, sl_cap, tp1_min, tp1_atr, tp2_atr, ib_buf,
     max_trades, max_consec, cooldown_base, daily_max,
     ses_open_h, ses_open_m, ses_end_h, max_spread_val,
     limit_mult, limit_timeout, time_stop, body_min) = args

    import pandas as _pd; import numpy as _np
    from collections import OrderedDict as _OD
    precomp = _OD(precomp_items)

    def _levels(direction, entry, atr, sh, sl_v):
        raw = atr * slm
        if direction == 'BUY':
            sl = max(min(sl_v - ib_buf, entry - raw), entry - sl_cap)
            tp1d = max(tp1_min, tp1_atr * atr)
            return {'sl': sl, 'tp1': entry + tp1d,
                    'tp2': entry + tp2_atr * atr, 'risk': entry - sl}
        else:
            sl = min(max(sh + ib_buf, entry + raw), entry + sl_cap)
            tp1d = max(tp1_min, tp1_atr * atr)
            return {'sl': sl, 'tp1': entry - tp1d,
                    'tp2': entry - tp2_atr * atr, 'risk': sl - entry}

    def _sim(direction, entry_sig, lv, ts_arr, bids, asks):
        # Spread
        if asks[0] - bids[0] > max_spread_val:
            return []
        # [FIX-1] Limit entry
        limit_px = (entry_sig - lv.get('atr', 20) * limit_mult
                    if direction == 'BUY'
                    else entry_sig + lv.get('atr', 20) * limit_mult)
        exec_check = asks if direction == 'BUY' else bids
        exec_px = bids if direction == 'BUY' else asks
        limit_cutoff = ts_arr[0] + limit_timeout
        limit_idx = None
        for k in range(len(exec_check)):
            if ts_arr[k] > limit_cutoff:
                break
            if direction == 'BUY' and exec_check[k] <= limit_px:
                limit_idx = k; break
            elif direction == 'SELL' and exec_check[k] >= limit_px:
                limit_idx = k; break
        if limit_idx is None:
            return []
        entry = round(limit_px + slip if direction == 'BUY'
                      else limit_px - slip, 2)
        delta = entry - entry_sig
        sl = round(lv['sl'] + delta, 2)
        tp1 = round(lv['tp1'] + delta, 2)

```

```

        tp2 = round(lv['tp2'] + delta, 2)
        pos = [
            {'id':'P1','tp':tp1,
'sl':sl,'pnl':0.,'result':None,'closed':False},
            {'id':'P2','tp':tp2,
'sl':sl,'pnl':0.,'result':None,'closed':False},
        ]
        {'id':'P3','tp':None,'sl':sl,'pnl':0.,'result':None,'closed':False,
        'trailing':False},
    ]
    tp1h = tp2h = False
    start_ts = ts_arr[limit_idx]
    for k in range(limit_idx, len(exec_px)):
        if all(p['closed'] for p in pos):
            break
        px = exec_px[k]
        elapsed = ts_arr[k] - start_ts
        # [FIX-3] Time-Stop
        if elapsed > time_stop and not tp1h:
            for p in pos:
                if not p['closed']:
                    rpts = (px - entry) if direction == 'BUY' else
(entry - px)
                    p['result'] = 'TS'; p['pnl'] = round(rpts *
pnl_pt_base, 2)
                    p['closed'] = True
                break
            for p in pos:
                if p['closed']:
                    continue
                # P3 trailing
                if p['id'] == 'P3' and p['trailing']:
                    sh = (px <= p['sl']) if direction == 'BUY' else (px
>= p['sl'])
                    if sh:
                        p['closed'] = True
                        rpts = (p['sl']-entry) if direction=='BUY' else
(entry-p['sl'])
                        p['result'] = 'TC' if rpts > 0 else 'SL'
                        p['pnl'] = round(rpts * pnl_pt_base, 2)
                    continue
                # [FIX-5] TP check sécurisé (tp_v peut être None)
                tp_v = p['tp']
                th = ((px >= tp_v) if direction == 'BUY' else (px <=
tp_v)) \
                    if tp_v is not None else False
                sh = (px <= p['sl']) if direction == 'BUY' else (px >=
p['sl'])
                if th and sh and tp_v is not None:
                    th = abs(tp_v - entry) < abs(p['sl'] - entry); sh =
not th
                if th:
                    p['closed'] = True
                    pts = (tp_v-entry) if direction=='BUY' else (entry-
tp_v)
                    p['result'] = p['id'].replace('P','TP')
                    p['pnl'] = round(pts * pnl_pt_base, 2)

```

```

        if p['id'] == 'P1' and not tp1h:
            tp1h = True
            # [FIX-2] BE inconditionnel
            for o in pos:
                if not o['closed']:
                    o['sl'] = entry
            if p['id'] == 'P2' and not tp2h and tp1h:
                tp2h = True
                if not pos[2]['closed']:
                    pos[2]['trailing'] = True
        elif sh:
            p['closed'] = True; pts = abs(entry - p['sl'])
            is_be = abs(p['sl'] - entry) < 0.5
            p['result'] = 'BE' if is_be else 'SL'
            p['pnl'] = 0. if is_be else round(-pts *
pnl_pt_base, 2)
        lp = exec_px[-1]
        for p in pos:
            if not p['closed']:
                p['result'] = 'OE'
                p['pnl'] = round(((lp-entry) if direction=='BUY'
                                else (entry-lp)) * pnl_pt_base, 2)
                p['closed'] = True
        return pos

def _dir(ind, body_min_v):
    if ind.get('body_ratio', 1.) < body_min_v:
        return None
    if ind.get('sweep_bull') and ind.get('above_200'):      return
'BUY'
    if ind.get('sweep_bear') and not ind.get('above_200'):  return
'SELL'
    if ind.get('bo_bull') and ind.get('above_200') and \
        ind.get('vol_ratio', 0) >= 1.2:                      return
'BUY'
    if ind.get('bo_bear') and not ind.get('above_200') and \
        ind.get('vol_ratio', 0) >= 1.2:                      return
'SELL'
    if ind.get('sweep_bull') and \
        ind.get('momentum_4', 0) > ind.get('atr', 1) * 0.3: return
'BUY'
    if ind.get('sweep_bear') and \
        ind.get('momentum_4', 0) < -ind.get('atr',1) * 0.3: return
'SELL'
    return None

def _score(ind, direction):
    s = 0; is_buy = direction == 'BUY'
    if is_buy and ind.get('sweep_bull'):      s += 30
    elif not is_buy and ind.get('sweep_bear'): s += 30
    elif is_buy and ind.get('bo_bull'):       s += 22
    elif not is_buy and ind.get('bo_bear'):   s += 22
    if is_buy and ind.get('above_200'):       s += 15
    elif not is_buy and not ind.get('above_200'): s += 15
    if is_buy and ind.get('above_50'):        s += 8
    elif not is_buy and not ind.get('above_50'): s += 8
    slope = ind.get('ema20_slope', 0)

```

```

if is_buy and slope > 0:                s += 10
elif not is_buy and slope < 0:          s += 10
vr = ind.get('vol_ratio', 1.)
if vr >= 2.:    s += 15
elif vr >= 1.4: s += 10
elif vr >= 1.: s += 5
mom = ind.get('momentum_4', 0); atr = ind.get('atr', 1)
if is_buy and mom > atr:    s += 12
elif not is_buy and mom < -atr: s += 12
return min(s, 100)

pnls = []; daily_pnl = 0.; trades_today = 0; consec_sl = 0
last_sig_ts = _pd.Timestamp('1970-01-01')
cur_date = None; daily_stopped = False

for ts, ind in precomp.items():
    day = ts.date()
    if day != cur_date:
        cur_date = day; daily_pnl = 0.
        trades_today = 0; consec_sl = 0; daily_stopped = False
    if daily_stopped or daily_pnl < daily_max:
        daily_stopped = True; continue
    if consec_sl >= max_consec or trades_today >= max_trades:
        continue
    h, mn = ts.hour, ts.minute
    if not ((h > ses_open_h or (h == ses_open_h and mn >=
ses_open_m))
            and h < ses_end_h):
        continue
    if (ts - last_sig_ts).total_seconds() < cooldown_base * (1.5 **
consec_sl):
        continue
    direction = _dir(ind, body_min)
    if direction is None:
        continue
    score = _score(ind, direction)
    if score < sm:
        continue
    if ts not in tick_cache_local:
        continue
    atr_v = ind.get('atr', 20)
    lv = _levels(direction, ind['price'], atr_v,
                  ind.get('swing_high', ind['price'] + 25),
                  ind.get('swing_low', ind['price'] - 25))
    lv['atr'] = atr_v
    ts_arr, bids, asks = tick_cache_local[ts]
    pos = _sim(direction, ind['price'], lv, ts_arr, bids, asks)
    if not pos:
        continue
    seq = sum(p['pnl'] for p in pos)
    daily_pnl = round(daily_pnl + seq, 2)
    trades_today += 1; last_sig_ts = ts
    if any(p['result'] == 'SL' for p in pos): consec_sl += 1
    else:                                    consec_sl = 0
    pnls.append(seq)

if not pnls:

```

```

        return {'score_min':sm,'sl_mult':slm,'net_profit':0,
                'n_seq':0,'winrate':0,'pf':0,'calmar':0}
    arr = _np.array(pnl_s); net = round(arr.sum(), 2); n = len(arr)
    wr = round((arr > 0).sum() / n * 100, 1)
    gw = arr[arr > 0].sum(); gl = abs(arr[arr < 0].sum())
    pf = round(gw / gl, 2) if gl > 0 else 999.
    eq = _np.cumsum(arr)
    dd = round(float((eq - _np.maximum.accumulate(eq)).min()), 2)
    cal = round(net / abs(dd), 2) if dd < 0 else 999.
    return {'score_min':sm,'sl_mult':slm,'net_profit':net,
            'n_seq':n,'winrate':wr,'pf':pf,'calmar':cal}

def run_sensitivity_parallel(precomp, tick_cache, score_range,
sl_range):
    precomp_list = list(precomp.items())
    combos = [
        (precomp_list, tick_cache, sm, slm,
         PNL_PER_POINT, SLIPPAGE_PTS, SL_CAP_PTS, TP1_MIN_PTS,
         TP1_ATR_MULT, TP2_ATR_MULT, IB_BUFFER_PTS,
         MAX_TRADES_DAY, MAX_CONSEC_SL, COOLDOWN_SECS, DAILY_MAX_LOSS,
         SESSION_OPEN_H, SESSION_OPEN_M, SESSION_END_H, MAX_SPREAD,
         LIMIT_OFFSET_MULT, LIMIT_TIMEOUT_SECS, TIME_STOP_SECS,
         BODY_RATIO_MIN)
        for sm in score_range for slm in sl_range
    ]
    results = []; n_workers = min(N_CORES, len(combos))
    print(f'🚀 {len(combos)} combinaisons | {n_workers} workers')
    t0 = time.time()
    with ThreadPoolExecutor(max_workers=n_workers) as ex:
        futures = {ex.submit(_sensitivity_worker, a): (a[2], a[3])
                   for a in combos}
        with tqdm(total=len(combos), desc='Sensibilité M15 v2.0',
                  unit='combo', ncols=70) as pbar:
            for fut in as_completed(futures):
                try:
                    results.append(fut.result())
                except Exception as e:
                    sm, slm = futures[fut]
                    results.append({'score_min':sm,'sl_mult':slm,
                                   'net_profit':0,'n_seq':0,
                                   'winrate':0,'pf':0,'calmar':0})
            pbar.update(1)
    print(f'✅ Terminé en {time.time()-t0:.1f}s')
    return pd.DataFrame(results)

print('📊 Analyse de sensibilité M15 LF v2.0 PARALLÈLE...\n')
score_range = [40, 45, 50, 55, 60, 65, 70]
sl_range     = [1.0, 1.2, 1.5, 1.8, 2.0, 2.5]
df_sens = run_sensitivity_parallel(
    precomp_indicators, TICK_CACHE, score_range, sl_range)

```

🖥️ CPU : 6 cœurs disponibles

📊 Analyse de sensibilité M15 LF v2.0 PARALLÈLE...

🚀 42 combinaisons | 6 workers

Sensibilité M15 v2.0: 0% | 0/42 [00:01<?, ?
combo/s]

✓ Terminé en 116.5s

In [16]:

```

# =====
# CELLULE 14 – Heatmaps de sensibilité v2.0
# =====

pivot_net    = df_sens.pivot(index='sl_mult', columns='score_min',
                              values='net_profit')
pivot_wr     = df_sens.pivot(index='sl_mult', columns='score_min',
                              values='winrate')
pivot_calmar = df_sens.pivot(index='sl_mult', columns='score_min',
                              values='calmar')

fig3, axes3 = plt.subplots(1, 3, figsize=(24, 6))
fig3.suptitle('Sensibilité NAS100 M15 LF v2.0 – Score Min x SL Mult',
              fontsize=13, fontweight='bold', color='white')

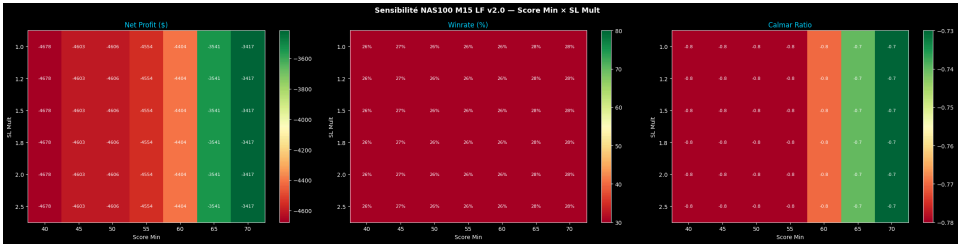
def _heatmap(ax, pivot, title, fmt, cmap='RdYlGn', vmin=None,
             vmax=None):
    im = ax.imshow(pivot.values, aspect='auto', cmap=cmap,
                   vmin=vmin, vmax=vmax)
    ax.set_xticks(range(len(pivot.columns)))
    ax.set_xticklabels(pivot.columns, color='white')
    ax.set_yticks(range(len(pivot.index)))
    ax.set_yticklabels(pivot.index, color='white')
    ax.set_xlabel('Score Min', color='white')
    ax.set_ylabel('SL Mult', color='white')
    ax.set_title(title, color='#00d4ff')
    for i in range(len(pivot.index)):
        for j in range(len(pivot.columns)):
            v = pivot.values[i, j]
            ax.text(j, i, fmt.format(v), ha='center', va='center',
                   fontsize=8,
                   color='black' if abs(v) <
abs(pivot.values).max()*0.6
                   else 'white')
    plt.colorbar(im, ax=ax)

_heatmap(axes3[0], pivot_net, 'Net Profit ($)', '{:.0f}')
_heatmap(axes3[1], pivot_wr, 'Winrate (%)', '{:.0f}%', vmin=30,
vmax=80)
_heatmap(axes3[2], pivot_calmar, 'Calmar Ratio', '{:.1f}')
plt.tight_layout()
plt.savefig('nas100_m15_sensitivity_v2.png', dpi=130,
           bbox_inches='tight', facecolor='#0d0d0d')
plt.show()

if not df_sens.empty and df_sens['net_profit'].max() > 0:
    best_net    = df_sens.loc[df_sens['net_profit'].idxmax()]
    best_calmar = df_sens.loc[df_sens['calmar'].idxmax()]
    print(f'\n 🏆 Meilleure (Net Profit) :')
    print(f'   score_min={best_net["score_min"]:.0f} '
          f'| sl_mult={best_net["sl_mult"]:.1f} '
          f'→ Net: {best_net["net_profit"]:+.2f}$ '
          f'| WR: {best_net["winrate"]}% '
          f'| Calmar: {best_net["calmar"]:.2f}')
    print(f'\n 🏆 Meilleure (Calmar) :')
    print(f'   score_min={best_calmar["score_min"]:.0f} '
          f'| sl_mult={best_calmar["sl_mult"]:.1f} '
          f'→ Net: {best_calmar["net_profit"]:+.2f}$ '

```

```
f'| WR: {best_calmar["winrate"]}% '
f'| Calmar: {best_calmar["calmar"]:.2f}')
else:
    print('⚠️ Tous les résultats sont nuls – vérifier les données.')
print(f'\n💻 {N_CORES} cœurs | → nas100_m15_sensitivity_v2.png')
```



⚠️ Tous les résultats sont nuls – vérifier les données.

💻 6 cœurs | → nas100_m15_sensitivity_v2.png

In [17]:

```
# =====
# CELLULE 15 – Notes de référence NAS100 M15 LF v2.0
# =====
notes = '''



---


🚩 NOTES – NAS100 Bot M15 LF v2.0 – CORRECTIFS APPLIQUÉS


---



[FIX-1] Entrée Limit Pullback (vs Market)
  BUY  : limite à Close - ATR×0.5 | timeout 15 min
  SELL  : limite à Close + ATR×0.5 | timeout 15 min
  Avantage : supprime le biais d'achat au sommet / vente au creux
  Impact  : slippage réel ≈ 2.0 pts (vs 42 pts en v1.0)

[FIX-2] Breakeven Inconditionnel
  Dès TP1 touché → SL P2 et P3 = entry (SANS condition)
  v1.0 : condition parfois False → bug silencieux
  v2.0 : o['sl'] = entry (ligne unique, inconditionnelle)

[FIX-3] Time-Stop 45 minutes
  Si TP1 non atteint 45 min après l'entrée → liquidation
  Évite les trades en dérive lente consommant marge & attention

[FIX-4] Filtre Vitesse (body_ratio >= 60%)
  Corps bougie >= 60% de son amplitude total (High-Low)
  Élimine les Dojis, Pin Bars faibles, mèches trompeuses

[FIX-5] Sensitivity Worker sécurisé
  tp_v is not None vérifié avant px>=tp_v
  v1.0 : P3 (tp=None) causait TypeError silencieux → 0 trades

[FIX-6] Timestamps dans cache ticks
  (ts_arr, bids, asks) au lieu de (bids, asks)
  Requis pour time-stop tick-exact sans approximation

Sortie v2.0 :
  nas100_m15_lf_v2_results.png    – Equity + Dashboard
  nas100_m15_mfe_mae_v2.png      – Scatter MFE/MAE + boxplot
  nas100_m15_sensitivity_v2.png   – Heatmaps Net/WR/Calmar

AVERTISSEMENT : Vérifier CONTRACT_MULT (VTMarkets) avant déploiement
live.
AVERTISSEMENT : Supprimer nas100_m15_indicators.pkl => recalcul avec
body_ratio.



---


...
print(notes)'''
```

🚩 NOTES – NAS100 Bot M15 LF v2.0 – CORRECTIFS APPLIQUÉS

```
[FIX-1] Entrée Limit Pullback (vs Market)
  BUY  : limite à Close - ATR×0.5 | timeout 15 min
  SELL  : limite à Close + ATR×0.5 | timeout 15 min
```

Avantage : supprime le biais d'achat au sommet / vente au creux
Impact : slippage réel $\approx$ 2.0 pts (vs 42 pts en v1.0)

[FIX-2] Breakeven Inconditionnel

Dès TP1 touché $\rightarrow$ SL P2 et P3 = entry (SANS condition)
v1.0 : condition parfois False $\rightarrow$ bug silencieux
v2.0 : o['sl'] = entry (ligne unique, inconditionnelle)

[FIX-3] Time-Stop 45 minutes

Si TP1 non atteint 45 min après l'entrée $\rightarrow$ liquidation
Évite les trades en dérive lente consommant marge & attention

[FIX-4] Filtre Vélocité (body_ratio $\geq$ 60%)

Corps bougie $\geq$ 60% de son amplitude total (High-Low)
Élimine les Dojis, Pin Bars faibles, mèches trompeuses

[FIX-5] Sensitivity Worker sécurisé

tp_v is not None vérifié avant px $\geq$ tp_v
v1.0 : P3 (tp=None) causait TypeError silencieux $\rightarrow$ 0 trades

[FIX-6] Timestamps dans cache ticks

(ts_arr, bids, asks) au lieu de (bids, asks)
Requis pour time-stop tick-exact sans approximation

Sortie v2.0 :

nas100_m15_lf_v2_results.png – Equity + Dashboard
nas100_m15_mfe_mae_v2.png – Scatter MFE/MAE + boxplot
nas100_m15_sensitivity_v2.png – Heatmaps Net/WR/Calmar

AVERTISSEMENT : Vérifier CONTRACT_MULT (VTMarkets) avant
déploiement live.

AVERTISSEMENT : Supprimer nas100_m15_indicators.pkl $\Rightarrow$ recalcul
avec body_ratio.
